

Fashion Swap — Clothing Exchange & Swap Marketplace

Project Title: Clothing Exchange & Swap Marketplace

Product Name: Fashion Swap

Version: 1.0 — Phase 1

Project Type: Full-Stack Web Application

Technology Stack: React, Vite, Node.js, Express.js, MongoDB, JWT

Document Type: Final Project Report

Repository Structure: Monorepo (`client/` + `server/`)

Prepared For: Unified Mentor Internship Project Evaluation

Abstract

Fashion Swap is a full-stack web application designed to enable users to exchange wearable clothing through a direct barter model instead of traditional buying and selling.

The platform addresses the problem of unused clothing and fast-fashion waste by providing users with a dedicated marketplace where they can create clothing listings, browse available items, search and filter listings, discover location-based swap opportunities, send swap requests, compare estimated item values, negotiate through chat, and complete clothing exchanges.

The application follows a modern client-server architecture using React and Vite for the frontend, Node.js and Express.js for the backend, and MongoDB with Mongoose for persistent data storage. JWT-based authentication and bcrypt password hashing are used to secure user accounts. The system also includes role-based administrative functionality for managing users, listings, and swap activity.

Phase 1 implements the core Clothing Exchange Marketplace requirements specified for the Unified Mentor project, including authentication, clothing listings, marketplace browsing, item details, swap requests, negotiation messaging, value comparison, location-based filtering, user dashboard functionality, and an administrative panel.

The application was manually tested across the major user workflows and interconnected modules. Production deployment is planned using MongoDB Atlas, Render, and Vercel.

1. Introduction

Fast fashion has significantly increased clothing consumption and contributed to growing textile waste. Many people own wearable clothing that they no longer use but may not have a convenient way to exchange or redistribute it.

Traditional e-commerce platforms primarily focus on buying and selling. Resale platforms introduce monetary pricing, while donation platforms do not provide users with a direct exchange mechanism.

Fashion Swap addresses this gap by providing a dedicated clothing exchange platform based on a barter model.

Users can:

- Create personal accounts.
- Build profiles.
- Upload clothing items.
- Add clothing information such as brand, category, size, condition, and location.
- Browse and search marketplace listings.
- Filter items based on relevant attributes.
- Compare estimated swap values.
- Send swap requests.
- Negotiate through chat.
- Choose local or courier-based exchange preferences.
- Track swap activity.

Administrators can monitor users, listings, and swap activity through a dedicated admin panel.

2. Problem Statement

The project was developed to address the following problems:

Problem	Impact
Many people own unused wearable clothing	Clothing remains unused or may eventually become waste
Traditional e-commerce focuses on buying and selling	Users need to handle monetary transactions
Resale platforms require pricing	Pricing can create friction for casual exchanges
Few platforms focus specifically on clothing swaps	Users have limited dedicated barter options
Matching without location context can be inconvenient	Local exchanges become more difficult
Users may disagree about item value	A value comparison mechanism can assist negotiations

Fashion Swap addresses these problems through a:

List → Discover → Compare → Request → Negotiate → Exchange

workflow.

3. Project Objectives

3.1 Primary Objectives

The primary objectives of Fashion Swap are:

- Provide a dedicated marketplace for clothing exchange.
- Enable users to exchange clothing without monetary transactions in Phase 1.
- Encourage clothing reuse and sustainable fashion.
- Provide location/city-based discovery of swap opportunities.
- Simplify the process of initiating and managing clothing swaps.

3.2 Secondary Objectives

The secondary objectives are:

- Provide negotiation messaging between users.
 - Estimate clothing swap values.
 - Compare the estimated values of offered and requested items.
 - Allow filtering by category, size, condition, and location.
 - Allow users to specify local or courier exchange preferences.
 - Provide administrative monitoring and moderation capabilities.
 - Create a foundation for future sustainability and community features.
-

4. Requirement Analysis

The project was developed according to the requirements supplied for the Unified Mentor Clothing Exchange & Swap Marketplace project.

Unified Mentor Requirement	Implementation Status
6-8 interconnected application pages	Completed
Login and registration	Completed
User dashboard	Completed
Clothing listing system	Completed
Item details page	Completed
Swap request system	Completed
Negotiation chat	Completed
Swap value calculator	Completed
Location-based matching/filtering	Completed
Admin panel	Completed

Unified Mentor Requirement	Implementation Status
Secure authentication	Completed
Responsive design	Completed
Realistic clothing data	Completed
Validation	Completed
Database persistence	Completed
Manual integration testing	Completed
Live deployment	To be completed/verified before final submission

The application contains more than the minimum requested page count because additional pages such as Profile, Add Item, Swaps, and Dashboard were implemented to provide a complete user experience.

5. Scope of Work

5.1 In Scope — Phase 1

The following functionality is included:

- User registration and login.
- User profile management.
- Clothing listing creation.
- Clothing image uploads.
- Marketplace browsing.
- Search and filtering.
- Item detail pages.
- Location/city-based filtering.
- Swap request creation.
- Incoming and outgoing swap management.
- Swap value comparison.
- Negotiation messaging.
- Swap status management.
- User dashboard.
- Administrative management.
- Responsive interface.

5.2 Out of Scope — Phase 1

The following features are intentionally deferred:

- Online payment systems.
- Wallet functionality.
- AI-powered recommendations.
- AR/virtual try-on.
- Native mobile applications.

- Automated courier/shipping integration.
- Advanced dispute arbitration.
- Automated clothing-condition verification.

6. Comparison With Related Platforms

Platform	Primary Focus	Difference From Fashion Swap
Depop	Community fashion marketplace	Primarily commerce/resale oriented
Poshmark	Fashion resale marketplace	Primarily payment and resale oriented
Swap.com	Clothing exchange	Similar exchange concept
Fashion Swap	Clothing exchange	Focuses on direct barter, value comparison, location filtering, and swap workflow

Fashion Swap focuses specifically on **clothing-to-clothing exchange**, rather than conventional buying and selling.

7. Functional Requirements

7.1 User Module

The user module provides:

- Registration.
- Login.
- Profile management.
- Location/city information.
- Bio and avatar.
- Password management.
- Clothing listing management.
- Swap management.
- Chat access.

7.2 Clothing Listing Module

Users can:

- Upload clothing images.
- Add title and description.
- Specify brand.
- Select clothing category.

- Select size.
- Specify condition.
- Add location/city.
- View estimated swap value.
- Edit listings.
- Remove listings.
- Track listing availability.

Listing statuses include:

- available
- pending
- swapped
- removed

7.3 Marketplace Module

The marketplace provides:

- Clothing listing grid.
- Search.
- Category filtering.
- Size filtering.
- Condition filtering.
- Location filtering.
- Nearby/location-based filtering.
- Pagination.
- Item detail navigation.
- Featured/similar item discovery.

7.4 Swap Request Module

The swap workflow allows a user to:

1. Select another user's clothing item.
2. Select one of their own available items.
3. Submit a swap request.
4. View estimated value comparison.
5. Negotiate the exchange.
6. Accept or reject the request.
7. Complete or cancel the swap.

Supported statuses include:

pending → accepted / rejected / negotiating → completed / cancelled

The exact status transitions are controlled by the backend implementation.

7.5 Negotiation Chat Module

The chat system allows users involved in a swap to communicate about:

- Item condition.
- Exchange details.
- Value differences.
- Local meeting arrangements.
- Courier preferences.
- Final exchange agreement.

Messages are associated with the relevant swap request so that conversations remain connected to the correct exchange.

The Phase 1 implementation uses API-based messaging. The backend includes infrastructure suitable for future realtime Socket.io communication.

7.6 Swap Value Calculator

Fashion Swap includes a custom estimated-value calculation system.

The calculation follows the general model:

Estimated Value = Category Base × Condition Multiplier + Brand Bonus

A minimum value floor is applied.

The system compares the estimated values of the requested and offered items and provides a fairness indication.

The fairness comparison treats a difference of up to approximately 20% relative to the higher estimated value as reasonably balanced.

The value is an estimate intended to assist negotiation rather than represent a real monetary market price.

7.7 Location-Based Matching

The marketplace provides location-aware discovery using user/listing location and city information.

Users can enable location-based filtering to find clothing opportunities associated with their location.

Phase 1 uses **city/location-based matching** rather than a full GPS-distance or map-based recommendation system.

7.8 Admin Module

The admin panel provides administrative functionality for:

- Viewing users.
- Monitoring listings.
- Managing inappropriate listings.
- Monitoring swap activity.
- Viewing platform statistics.
- Managing user activation status.

A dedicated admin authorization layer prevents normal users from accessing administrator functionality.

8. Non-Functional Requirements

The application addresses the following non-functional requirements:

Security

- JWT authentication.
- bcrypt password hashing.
- Protected API routes.
- Role-based authorization.
- Environment-based secrets.
- CORS configuration.

Performance

- Pagination for marketplace results.
- Modular API architecture.
- Database indexing for commonly queried fields.
- Optimized frontend component structure.

Usability

- Responsive layouts.
- Clear marketplace navigation.
- Structured swap workflow.
- User-friendly forms.
- Feedback through UI notifications.

Scalability

The client and server are separated so that frontend and backend services can be deployed independently.

Privacy

Sensitive configuration values are stored through environment variables rather than being committed to the source repository.

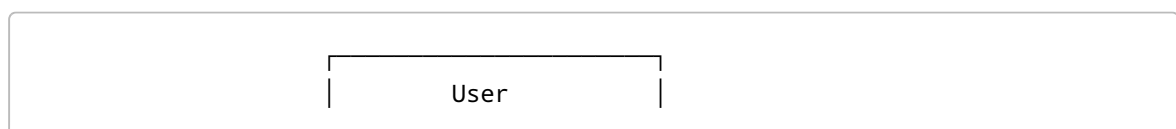
9. Technology Stack

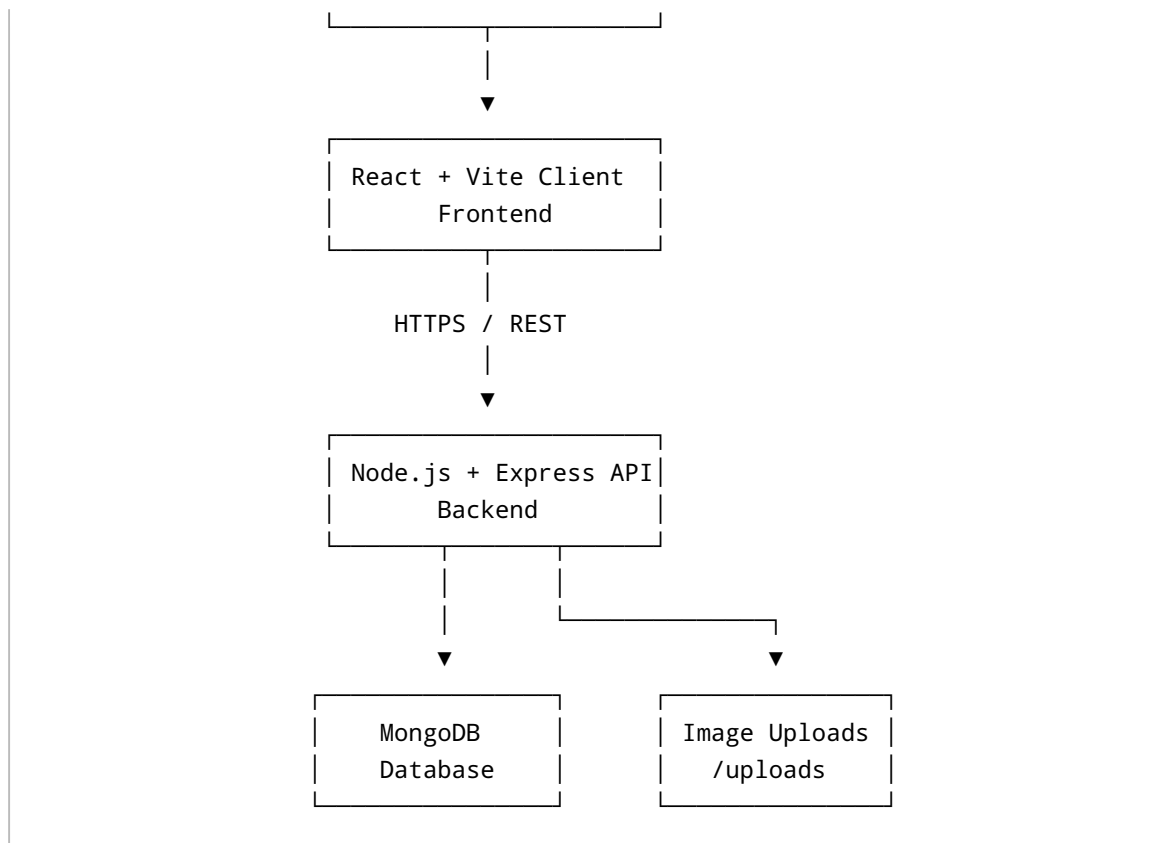
Layer	Technology
Frontend	React
Build Tool	Vite
Routing	React Router
Styling	CSS / project styling system as implemented
HTTP Client	Axios
Notifications	React Hot Toast
Animation	Framer Motion
Backend	Node.js
API Framework	Express.js
Database	MongoDB
ODM	Mongoose
Authentication	JSON Web Token
Password Security	bcryptjs
File Uploads	Multer
Realtime Infrastructure	Socket.io-ready server
Development	Nodemon
Deployment Target	Vercel + Render + MongoDB Atlas

The final technology list should match the actual dependencies present in `package.json`.

10. System Architecture

Fashion Swap follows a client-server architecture.





11. Frontend Architecture

The React application is organized into reusable components, pages, contexts, services, and routes.

Major frontend responsibilities include:

- Rendering marketplace content.
- Managing forms.
- Handling authentication state.
- Calling backend APIs.
- Protecting private routes.
- Displaying swap information.
- Displaying chat messages.
- Managing responsive layouts.
- Providing administrator interfaces.

The frontend uses an environment variable for the backend API URL:

```
VITE_API_URL=http://localhost:5000/api
```

The production value is changed to the deployed backend URL.

12. Backend Architecture

The backend uses Node.js and Express.js.

Major API areas include:

```
/api/auth  
/api/listings  
/api/swaps  
/api/chat  
/api/admin
```

The backend includes:

- Controllers.
- Routes.
- Mongoose models.
- Authentication middleware.
- Authorization middleware.
- Upload middleware.
- Database configuration.
- Utility functions.
- Seed data.

The backend is responsible for:

- Authentication.
- Authorization.
- Data validation.
- Database operations.
- Swap processing.
- Value calculation.
- Messaging.
- Administrative operations.

13. Data Flow — Typical Swap

```
User  
↓  
Marketplace  
↓  
View Item  
↓  
Select "Swap"  
↓  
Select Owned Item
```

```
↓
Create Swap Request
↓
Backend Calculates Value Comparison
↓
Owner Receives Request
↓
Negotiation Chat
↓
Accept / Reject / Continue Negotiation
↓
Complete Swap
↓
Listing Status Updated
```

This workflow connects the marketplace, listing, swap, value-calculation, chat, and database modules.

14. Module Architecture

14.1 Authentication

Registration validates required information and creates a user account.

Passwords are hashed before storage.

Login verifies the credentials and returns a JWT.

The token is then used to access protected API resources.

14.2 Profile Management

Users can maintain:

- Name.
- Email.
- Phone.
- Location.
- City.
- Bio.
- Avatar.
- Password.

Password changes require verification of the current password.

14.3 Listing Management

Authenticated users can create and manage their clothing listings.

Listings contain:

- Title.
 - Description.
 - Brand.
 - Category.
 - Size.
 - Condition.
 - Estimated value.
 - Images.
 - Location.
 - Tags.
 - Owner.
 - Availability status.
-

14.4 Swap Management

Each swap connects:

- Requester.
- Owner.
- Requested item.
- Offered item.
- Value comparison.
- Exchange method.
- Swap status.

This structure allows the application to maintain a complete swap lifecycle.

14.5 Chat

Messages are connected to individual swap requests.

Each message contains:

- Swap request.
 - Sender.
 - Receiver.
 - Content.
 - Read status.
 - Timestamp.
-

14.6 Administration

The administrator can monitor:

- Users.
- Listings.
- Swaps.
- Platform statistics.

Administrative routes require appropriate authentication and authorization.

15. Database Design

MongoDB is used as the primary database.

15.1 User Collection

Field	Type / Description
fullName	String
email	String, unique
password	Hashed String
phone	String
location	String
city	String
bio	String
avatar	String
role	user/admin
isActive	Boolean
swapCount	Number
timestamps	createdAt / updatedAt

15.2 Listing Collection

Field	Type / Description
title	String
description	String

Field	Type / Description
brand	String
category	String
size	String
condition	String
estimatedValue	Number
images	Array
location	String
city	String
owner	ObjectId → User
status	Enum
tags	Array
timestamps	createdAt / updatedAt

Indexes are used for relevant search and filtering operations.

15.3 SwapRequest Collection

Field	Type / Description
requester	ObjectId → User
owner	ObjectId → User
requestedItem	ObjectId → Listing
offeredItem	ObjectId → Listing
message	String
status	Enum
valueComparison	Embedded object
exchangeMethod	local/courier/undecided

15.4 Message Collection

Field	Type / Description
swapRequest	ObjectId → SwapRequest
sender	ObjectId → User

Field	Type / Description
receiver	ObjectId → User
content	String
read	Boolean
timestamps	createdAt / updatedAt

An index on swap request and creation time supports chronological message retrieval.

16. Application Pages

Page	Route	Purpose
Home	/	Introduction, categories, featured items and CTA
Marketplace	/marketplace	Browse, search and filter listings
Item Details	/item/:id	View complete clothing information
Login	/login	User authentication
Register	/register	New account creation
Dashboard	/dashboard	User statistics and activity
Profile	/profile	Profile management
Add Item	/add-item	Create clothing listing
Swaps	/swaps	Manage swap requests
Swap Request	/swap-request/:itemId	Create a swap request
Chat	/chat	View conversations
Swap Chat	/chat/:swapId	Negotiate a specific swap
Admin	/admin	Administrative management

The project therefore exceeds the minimum requirement of 6–8 interconnected pages.

17. User Experience

The interface is designed around a modern fashion marketplace aesthetic.

The main UX principles include:

- Clear marketplace discovery.
- Visual clothing cards.

- Simple filtering.
- Easy access to item details.
- Clear swap calls-to-action.
- Value comparison during swaps.
- Dedicated negotiation conversations.
- Responsive navigation.
- Consistent spacing and typography.

The application is designed to support both desktop and mobile viewport sizes.

18. Security Implementation

Security Control	Implementation
Password storage	bcrypt hashing
Authentication	JWT
API protection	Authentication middleware
Admin protection	Role-based middleware
Frontend protection	Protected routes
Secrets	Environment variables
CORS	Configured client origins
Password change	Current-password verification
Database access	Mongoose models and server-side logic

Development/demo credentials should be treated as evaluation-only credentials and should not be used as production credentials.

19. Challenges Faced and Solutions

During development, several technical and architectural challenges were encountered.

19.1 Frontend-Backend Integration

Challenge:

The frontend and backend were separate applications and needed reliable API communication.

Solution:

The project was separated into `client/` and `server/` applications, with Axios used for API requests and environment variables used for API configuration.

Result:

The frontend and backend communicate through a modular REST API.

19.2 MongoDB Connectivity

Challenge:

The application needed to work with MongoDB during local development and support MongoDB Atlas for cloud deployment.

Solution:

The database connection was centralized and configured through the `MONGO_URI` environment variable.

Result:

The application can use different database environments without changing its core application logic.

19.3 Authentication and Authorization

Challenge:

The application required secure registration, login, protected pages, and administrator-only functionality.

Solution:

JWT authentication, bcrypt password hashing, protected routes, and admin authorization middleware were implemented.

Result:

User and administrator access are separated appropriately.

19.4 Complex Swap Workflow

Challenge:

A swap involves two users, two clothing items, estimated values, negotiation, and multiple statuses.

Solution:

A dedicated `SwapRequest` model was created to maintain all information related to an exchange.

Result:

The swap lifecycle can be tracked independently from individual clothing listings.

19.5 Swap Value Calculation

Challenge:

A fair comparison was required without depending on an external valuation service.

Solution:

A custom algorithm was created using category, condition, and brand information.

Result:

Users receive an estimated comparison to assist with negotiations.

19.6 Location-Based Matching

Challenge:

The project required location-aware discovery without introducing unnecessary GPS/map complexity in Phase 1.

Solution:

City and location information associated with profiles and listings is used for filtering.

Result:

Users can discover relevant local swap opportunities.

19.7 Image Uploads

Challenge:

Clothing and profile images needed to be uploaded through the application.

Solution:

Multer was implemented for multipart file handling and uploaded files are served through the backend.

Result:

Users can upload listing images and profile avatars.

Future consideration: Cloud media storage can be introduced for greater production scalability.

19.8 Chat and Negotiation

Challenge:

Users needed to communicate about their proposed swaps.

Solution:

Messages were associated with individual swap requests. API-based messaging was implemented, while the backend architecture can be extended with realtime Socket.io communication.

Result:

The required negotiation workflow is functional.

19.9 Admin Access

Challenge:

Normal users must not access administrator functionality.

Solution:

Backend authentication and admin authorization middleware were combined with frontend protected routes.

Result:

Administrative functions are restricted to authorized users.

19.10 Responsive Interface

Challenge:

The project contains marketplace cards, filters, dashboards, chat interfaces, and administrative views that need to work across screen sizes.

Solution:

Responsive layouts were implemented and tested across desktop and mobile viewport sizes.

Result:

The major application workflows remain usable across different devices.

19.11 Testing Interconnected Workflows

Challenge:

A change in one module can affect other modules. For example, creating a listing affects the marketplace, item details, and swap functionality.

Solution:

Manual integration testing was performed across complete workflows rather than only testing individual pages.

Result:

The interconnected Phase 1 functionality was verified.

20. Testing and Verification

The completed application was manually tested across its primary workflows.

#	Test Case	Expected Result	Status
1	API health check	API responds successfully	Passed
2	User registration	Account is created	Passed
3	User login	JWT authentication succeeds	Passed
4	Protected route	Unauthorized users are blocked	Passed

#	Test Case	Expected Result	Status
5	Profile update	User information updates successfully	Passed
6	Create clothing listing	Listing is stored and displayed	Passed
7	Edit/remove listing	Listing changes are reflected	Passed
8	Marketplace browsing	Listings load correctly	Passed
9	Search	Relevant listings are returned	Passed
10	Filters	Results update according to filters	Passed
11	Location filtering	Location-relevant listings are displayed	Passed
12	Item details	Correct listing information is displayed	Passed
13	Swap request	Request is created successfully	Passed
14	Value comparison	Offered/requested values are calculated	Passed
15	Incoming swap management	Owner can manage requests	Passed
16	Chat	Messages are saved and displayed	Passed
17	Swap status changes	Status changes correctly	Passed
18	Swap completion	Completed swap is recorded	Passed
19	Admin authentication	Admin can access admin area	Passed
20	Normal-user admin restriction	Unauthorized user is blocked	Passed
21	Admin user management	User status can be managed	Passed
22	Admin listing management	Listings can be moderated	Passed
23	Admin swap monitoring	Swap activity is visible	Passed
24	Responsive UI	Main screens work on mobile/desktop	Passed

Automated unit and end-to-end testing are identified as future improvements.

21. Deployment Strategy

The intended production architecture is:

```
Frontend → Vercel
Backend  → Render
Database → MongoDB Atlas
```

21.1 Database

MongoDB Atlas is intended for production database hosting.

21.2 Backend

The Express API is intended to be deployed as a Render Web Service.

Required environment variables include:

```
PORT=5000
MONGO_URI=<mongodb_uri>
JWT_SECRET=<strong_secret>
CLIENT_URL=<frontend_url>
```

21.3 Frontend

The React/Vite application is intended to be deployed through Vercel.

Production configuration:

```
VITE_API_URL=<deployed_backend_url>/api
```

21.4 Deployment Status

Status: To be completed and verified before final Unified Mentor submission.

Live Frontend URL:

```
<ADD LIVE VERCEL URL HERE>
```

Backend API URL:

```
<ADD LIVE RENDER URL HERE>
```

The final submission should only be made after the public frontend and backend have been tested successfully.

22. Local Development

Backend

```
cd server
npm install
npm run seed
npm run dev
```

Backend:

```
http://localhost:5000
```

Frontend

Open another terminal:

```
cd client  
npm install  
npm run dev
```

Frontend:

```
http://localhost:5173
```

The Vite port may change automatically if the default port is already occupied.

23. Project Structure

```
Fashion Swap/  
|  
├─ client/  
│   └─ src/  
│       ├── Pages/  
│       ├── components/  
│       ├── Context/  
│       ├── services/  
│       └─ routes/  
├─ package.json  
└─ ...  
  
├─ server/  
│   ├── config/  
│   ├── controllers/  
│   ├── models/  
│   ├── routes/  
│   ├── middleware/  
│   ├── utils/  
│   ├── seed.js  
│   ├── package.json  
│   └─ ...  
  
├─ PRD.md  
└─ DEPLOY.md
```

```
|— README.md
|— PROJECT_REPORT.md
|— .gitignore
```

The separation provides a clean distinction between frontend presentation and backend business/data logic.

24. Project Status

Component	Status
React frontend	Completed
Backend API	Completed
MongoDB integration	Completed
Authentication	Completed
User profiles	Completed
Clothing listings	Completed
Marketplace	Completed
Search and filters	Completed
Item details	Completed
Swap requests	Completed
Swap value calculation	Completed
Location filtering	Completed
Negotiation messaging	Completed
User dashboard	Completed
Admin panel	Completed
Responsive interface	Completed
Manual integration testing	Completed
Automated testing	Future enhancement
Cloud media storage	Future enhancement
Advanced realtime chat	Future enhancement
Production deployment	To be completed/verified

25. Limitations

Although the Phase 1 implementation satisfies the core marketplace and swap requirements, several limitations remain.

25.1 Estimated Value

Swap values are estimated using predefined rules and do not represent real market prices.

25.2 Clothing Condition

The platform relies on users to accurately describe the condition of their clothing.

25.3 Exchange Logistics

Users remain responsible for arranging physical exchanges.

The system records local/courier preferences but does not directly operate courier services.

25.4 Media Storage

Local filesystem image storage is suitable for development and evaluation but is less suitable for large-scale production.

25.5 Realtime Communication

The current messaging system uses API-based communication. A richer realtime experience can be introduced using Socket.io.

25.6 Dispute Resolution

Administrative monitoring is available, but a dedicated automated dispute arbitration system is outside Phase 1.

25.7 Payments

No monetary transactions or payment gateway are included because the Phase 1 product follows a barter model.

26. Future Enhancements

Potential future improvements include:

- AI-based swap recommendations.
- Advanced matching algorithms.
- GPS-based distance matching.
- Native mobile applications.

- Cloudinary or equivalent media storage.
 - Fully realtime chat.
 - Push notifications.
 - Clothing condition verification.
 - Sustainability impact tracking.
 - Community fashion groups.
 - User ratings and reviews.
 - Automated dispute resolution.
 - Courier/shipping API integration.
 - Optional payment or delivery-fee support.
-

27. Key Performance Indicators

The following KPIs can be used to evaluate the platform:

- Number of registered users.
- Number of active users.
- Number of clothing listings.
- Number of available listings.
- Number of swap requests.
- Number of completed swaps.
- Swap request conversion rate.
- User engagement.
- Number of active marketplace listings.

The Phase 1 admin dashboard provides basic operational statistics, while advanced analytics can be introduced in future versions.

28. Assumptions

The project assumes that:

- Users are willing to exchange wearable clothing.
 - Users provide accurate clothing information.
 - Users accurately describe item condition.
 - Users can independently arrange local exchanges.
 - Courier services can be used independently by users where required.
 - Users have access to internet-connected devices.
-

29. Constraints

The project has the following constraints:

- Clothing quality depends on user-provided information.
- Estimated swap value cannot guarantee actual market value.

- Physical exchanges are outside direct platform control.
 - Courier costs are handled independently by users.
 - Phase 1 does not include payment processing.
 - Local file storage has scalability limitations.
 - Automated testing is not included in Phase 1.
-

30. Expected Impact

Fashion Swap is designed to:

- Encourage clothing reuse.
- Reduce unnecessary clothing waste.
- Promote sustainable fashion behavior.
- Provide users with a cost-effective way to refresh their wardrobes.
- Encourage community-based exchange.
- Reduce dependence on conventional buy/sell transactions.
- Demonstrate practical implementation of a full-stack JavaScript application.

The project also demonstrates the complete development lifecycle:

Requirements → Design → Development → Database Integration → Authentication → Testing → Deployment Planning

31. Conclusion

Fashion Swap successfully implements a Phase 1 Clothing Exchange & Swap Marketplace based on the requirements provided for the Unified Mentor internship project.

The application provides the major required functionality, including secure authentication, user profiles, clothing listings, marketplace browsing, search and filtering, item details, location-based discovery, swap requests, value comparison, negotiation messaging, user dashboards, and administrative management.

The system uses a modular full-stack architecture with React and Vite on the client side, Node.js and Express.js on the server side, and MongoDB for persistent data storage.

The project was manually tested across its major interconnected workflows, including authentication, listing creation, marketplace discovery, swap requests, value comparison, messaging, swap status management, administrative functionality, and responsive behavior.

Certain advanced features—including AI recommendations, advanced realtime communication, automated courier integration, cloud media optimization, automated testing, and mobile applications—have intentionally been reserved for future phases.

The completed Phase 1 system provides a strong foundation for a sustainable fashion marketplace and demonstrates practical full-stack development skills, including frontend development, REST API design,

database management, authentication, authorization, file handling, business logic, testing, and deployment preparation.

32. References

1. Unified Mentor — Clothing Exchange & Swap Marketplace Project Requirements.
 2. Fashion Swap Product Requirements Document (`PRD.md`).
 3. Fashion Swap Project README (`README.md`).
 4. Fashion Swap Deployment Guide (`DEPLOY.md`).
 5. React Documentation — <https://react.dev>
 6. Express.js Documentation — <https://expressjs.com>
 7. MongoDB Documentation — <https://www.mongodb.com/docs>
 8. Mongoose Documentation — <https://mongoosejs.com>
 9. Depop — Community-driven fashion marketplace.
 10. Poshmark — Fashion resale marketplace.
 11. Swap.com — Clothing exchange marketplace.
-

Appendix A — Application Access

Standard User

Standard users should be created through the **Register** page.

Administrator

A development/evaluation administrator account may be provided separately to the evaluator.

Important: Production credentials and secrets must not be committed to the repository or publicly exposed in the project documentation.

Appendix B — Environment Variables

Server

```
PORT=5000
MONGO_URI=<mongodb_uri>
JWT_SECRET=<strong_secret>
CLIENT_URL=http://localhost:5173
```

Client

```
VITE_API_URL=http://localhost:5000/api
```

Production values must be configured through the respective hosting platform's environment-variable settings.

Appendix C — Final Submission Checklist

- ☒ Frontend completed.
 - ☒ Backend completed.
 - ☒ MongoDB integration completed.
 - ☒ Authentication implemented.
 - ☒ Clothing listing system completed.
 - ☒ Marketplace completed.
 - ☒ Search and filtering completed.
 - ☒ Item details completed.
 - ☒ Swap request workflow completed.
 - ☒ Swap value calculation completed.
 - ☒ Location-based filtering completed.
 - ☒ Negotiation messaging completed.
 - ☒ User dashboard completed.
 - ☒ Admin panel completed.
 - ☒ Responsive UI tested.
 - ☒ Major workflows manually tested.
 - ☐ Production frontend deployed.
 - ☐ Production backend deployed.
 - ☐ MongoDB Atlas configured for production.
 - ☐ Live URL verified.
 - ☐ Final GitHub repository cleaned of secrets.
 - ☐ Final project report updated with live URLs.
 - ☐ Final report exported to PDF.
-

Final Project: Fashion Swap — Clothing Exchange & Swap Marketplace

Version: 1.0 — Phase 1

Submission: Unified Mentor Internship Project